```
/Users/noel/.rbenv/versions/3.2.0/lib/ruby/gems/3.2.0/gems/rspec-3.12.0/lib/rspec.rb
```

If you're running `gem install` on a Unix platform and you aren't using a Ruby version manager, you'll need to prefix the command with `sudo`, because by default the local gems are installed into shared system directories.

The exact location will depend on the operating system you are using, plus which Ruby version manager you are using. In this case, the Ruby version manager maintains a hidden directory in my home directory for all the Ruby versions it is managing (/Users/noel/.rbenv/versions), and a specific subdirectory for the currently running version (/3.1.2). Inside that directory, the rest of the structure there is managed by Ruby gems, so there is a general set of directories for the version (/lib/ruby/gems/3.1.0/gems) and then a specific directory for this gem (/rspec-3.11.0) and then the path to the actual main script of the gem, which will typically install everything needed to use the gem (lib/rspec.rb).

Gem Version Numbers



You might wonder why the path to a gem file contains two apparently nearly-redundant folder names based on the version, in our case 3.1.2 and 3.1.0. The two directories are managed by different parts of the system. The outer 3.1.2 is managed by the Ruby version manager, and if you have multiple Ruby versions installed, you'll see that they each have different directories. This means that each version of Ruby gets its own separate copy of each gem. (It also means that you can reclaim some disc space by deleting versions of Ruby you don't use anymore.) The inner 3.1.0 is managed by Ruby itself and is the name of the directory that Ruby knows to use to look for gems. Each version of Ruby should have only one directory here.

Now the gem is ready to be used, which means that any Ruby program can `require rspec` and the RSpec library will be usable just as if it was part of the Ruby standard library. You'll need to check out the documentation for each gem to see how to use it. Often that documentation is accessible from the gem's home page.

The `gem` command has a lot of options, but the one you are most likely to use is `-v` or `--version`. When we installed RSpec, RubyGems installed the most recent version of the gem. Sometimes, you actually need a different gem version, usually for compatibility with a specific version, and you can specify the version string at the command line with `-v "3.0.0"` or whatever the actual version you need. You can have multiple versions of the same gem installed locally, but a general `require` statement will load the one with the highest version number. If you need to use a different version number, we recommend using Bundler.

With the gem installed, you can browse the source code locally with `gem open <GEMNAME>`, which opens the directory with that gem in your current system editor. You can edit the files in the gem, and because Ruby is an interpreted language, those edits will be available to other code that uses the gem, the interpreter will just load your changes. The most common case here is to place logging statements to aid in debugging. If you get too tangled up in your own edits, the command `gem pristine <GEMNAME>` will reinstall the original gem, and `gem uninstall <GEMNAME>` will just remove the gem.

Using Bundler to Manage Groups of Gems

Gems are pretty great, and any reasonably sized Ruby program will likely depend on many of them, each of which may have its own gem dependencies. This can easily become a